

מכונה תקשוב יחידה 3

אופרטורים וביטויים

איך גורמים ל Python לבצע חישובים בשבילנו — ולא רק לשמור ערכים כמו שהם.

תזכורת מהיחידה הקודמת

מה למדנו ביחידה 2?

- משתנה שומר ערך תחת שם, בעזרת סימן ההצבה =
- ארבעה טיפוסים בסיסיים: **bool**, **str**, **float**, **int**
- `type()` בודק מאיזה טיפוס משתנה מסוים
- Python משתמשת ב**הקלדה דינמית** — אפשר לשנות טיפוס בהמשך

טיפ. היום נלמד איך לשלב בין ערכים ומשתנים כדי לבצע איתם חישובים אמיתיים — לא רק לאחסן אותם.

מטרות השיעור

מה נלמד היום?

- מה זה **אופרטור**, ומה זה **ביטוי**
- אופרטורים אריתמטיים: חיבור, חיסור, כפל, חילוק ועוד
- חילוק שלם (`//`) ושארית חילוק (`%`)
- **סדר פעולות חשבון** ב Python — ומה קורה כששוכחים אותו
- **ההקשרים** בין אופרטורים — `(associativity)` מה קורה כשיש כמה אופרטורים באותה רמה

לפני שמתחילים

למה בכלל צריך אופרטורים?

עד עכשיו כל מה שעשינו הוא לשמור ערכים במשתנים ולהדפיס אותם כמו שהם. אבל תוכניות אמיתיות צריכות גם **לחשב** — לחבר שני מספרים, לחשב מחיר אחרי הנחה, לחלק תקציב בין אנשים. **אופרטורים** הם הכלי של Python לבצע חישובים כאלה, ו**ביטוי** הוא כל שילוב של ערכים ואופרטורים שמניב תוצאה.

טיפ. כל פעם שאתם רואים סימן כמו `+`, `-`, או `*` בין שני ערכים — זה אופרטור, וכל מה שהוא ומה שסביבו הוא ביטוי.

מושג מרכזי

מה זה ביטוי?

ביטוי (*expression*) הוא כל קטע קוד ש Python יכולה **לחשב לכדי ערך**. הביטוי הכי פשוט הוא ערך בודד, כמו 7.

אבל אפשר לחבר אליו אופרטורים כדי ליצור ביטויים מורכבים יותר. כשמריצים ביטוי Python, מחשבת אותו ומחזירה **תוצאה אחת** — בדיוק כמו במתמטיקה.

ארבעת הבסיסיים

האופרטורים האריתמטיים הבסיסיים

-חיסור

$7 \rightarrow 10 - 3$

+חיבור

$7 \rightarrow 3 + 4$

/חילוק

$4.5 \rightarrow 9 / 2$

***כפל**

$42 \rightarrow 6 * 7$

דוגמה

ארבעת האופרטורים בפעולה

נגדיר שני משתנים, ונפעיל עליהם את כל ארבעת האופרטורים:

```
a = 10
b = 4

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

פלט:

```
14
6
40
2.5
```

שימו לב

/ תמיד מחזיר float

גם כשהתוצאה "יוצאת עגולה" — "אופרטור החילוק הרגיל" / תמיד מחזיר float, אף פעם לא int.

9 / 3

3.0 — גם כשהתוצאה "עגולה"

8 / 2

4.0 — כן, עם נקודה!

תרגול מיידי

נסו עכשיו, לפני שממשיכים

קחו שתי דקות ונסו בעצמכם:

1. [קל] צרו שני משתנים a ו-b עם מספרים לבחירתכם, והדפיסו את הסכום שלהם.
2. [קל] חשבו כמה עולים 3 כרטיסים במחיר 45 ש"ח כל אחד, בעזרת אופרטור הכפל, והדפיסו את התוצאה.
3. [קל] הדפיסו את התוצאה של 4 / 10 — שימו לב לטיפוס שמתקבל.

חילוק שלם

// כמה פעמים שלמות האופרטור

לפעמים לא מעניינת אותנו הנקודה העשרונית — רק כמה פעמים שלמות מספר אחד "נכנס" בתוך השני. לשם כך יש אופרטור נפרד: // (חילוק שלם).

9 // 3

3 — כאן זה מסתדר בדיוק

7 // 2

3 — כמה פעמים 2 נכנס שלם בתוך 7

טיפ. שימו לב — התוצאה של // היא int, בניגוד לחילוק הרגיל / שתמיד מחזיר float.

שארית חילוק

האופרטור % — מה נשאר

מה עושים עם השארית שנשארת אחרי חילוק שלם? יש אופרטור שמחזיר בדיוק את זה: %, קוליה תיראש) modulo).

9 % 3

0 — כי 9 מתחלק בדיוק, בלי שארית

7 % 2

1 — כי $7 = 2 \times 3 + 1$

טיפ. שימוש נפוץ מאוד: 2 % n שווה 0 עבור מספר זוגי, ו-1 עבור אי-זוגי — נשתמש בזה כשנלמד תנאים ביחידה 6.

דוגמה // 10 %-יחד

לחלק פיצה בין חברים

מחלקים 17 פרוסות בין 5 חברים — כמה פרוסות שלמות לכל אחד, וכמה נשארות באמצע?

```
slices = 17
friends = 5

print(slices // friends) # לכל אחד
print(slices % friends) # נשארות
```

פלט:

3
2

תרגול

תרגול // ו-%

תרגילים שמשלבים חילוק שלם ושארית:

1. [קל] בהינתן `total_minutes = 130` (חשבו כמה שעות שלמות יש) `//` (וכמה דקות נשארות) `%`), והדפיסו את שניהם.

2. [קל] בדקו אם המספר 17 זוגי או אי-זוגי בעזרת `%`, על ידי הדפסת `2 % 17` — מה מתקבל?

3. [בינוני] חלקו 23 עפרונות בין 4 תלמידים בעזרת `//` ו-`%`, והדפיסו כמה מקבל כל תלמיד וכמה עפרונות נשארים.

עוד אופרטור

האופרטור ** — העלאה בחזקה

להעלאה בחזקה יש אופרטור משלה: `**` — לא `^` כמו שנהוג בכתב יד. הסימן `^` שמור ב-Python למשהו אחר לגמרי, שנפגוש ביחידה 8.

`5 ** 2`

25 — כלומר 5×5

`2 ** 3`

8 — כלומר $2 \times 2 \times 2$

דוגמה: `**`

שטח ריבוע בעזרת חזקה

ריבוע שאורך צלעו — 4 השטח הוא הצלע בריבוע:

```
side = 4
area = side ** 2
print(area)
```

פלט:

16

כלל חשוב

מה קורה כשמערבבים int ו-float-

מה קורה כששני הצדדים של אופרטור לא מאותו טיפוס? הכלל פשוט: אם אחד מהם — float התוצאה תהיה float, גם אם הצד השני היה int.

$3 + 2.0$

5.0 — אחד, float, גם התוצאה

$3 + 2$

5 — שני, int, תוצאה int

בדקו הבנה

נחשו לפני שמריצים

ענו קודם, ואז בדקו את עצמכם בהרצה בפועל:

1. [קל] מה הטיפוס של $4 * 2$ ומה הטיפוס של $4 * 2.0$?

2. [בינוני] בהינתן $x = 7$ ו- $y = 2$, מה התוצאה של x / y , ומה הטיפוס שלה?

3. [בינוני] בהינתן $a = 3$ ו- $b = 4$, חשבו את $a ** 2 + b ** 2$ בעצמכם על דף (משפט פיתגורס), ואז בדקו עם Python.

בדיוק כמו בבית הספר

סדר פעולות חשבון

מה קורה כשיש כמה אופרטורים באותו ביטוי? בדיוק כמו במתמטיקה שלמדתם בבית הספר — לכל אופרטור יש רמת עדיפות, ו Python מחשבת קודם את מה שברמה הגבוהה יותר.

טיפ. הכלל הבסיסי כבר מוכר לכם: קודם כפל וחילוק, ואז חיבור וחסור Python. לא ממציאה כאן שום דבר חדש.

טבלת עדיפויות

סדר העדיפויות המלא

· **1הכי גבוה:** () סוגריים

2: ** חזקה

3: % // * כפל, חילוק, חילוק שלם, שארית

· **4הכי נמוך:** - + חיבור, חיסור

מלכודת קלאסית

בלי לחשוב פעמיים — מה יודפס?

נסו לענות לפני שאתם ממשיכים לשקופית הבאה:

```
print(2 + 3 * 4)
```

פלט:

14

למה לא 20?

כפל לפני חיבור

גם אם קוראים את הביטוי משמאל לימין Python, לא מחשבת אותו בסדר הקריאה — היא מחשבת קודם את האופרטור עם העדיפות הגבוהה יותר.

שלב 2

12 + 2 → 14 (עכשיו החיבור)

שלב 1

4 * 3 → 12 (כפל קודם, כי עדיפותו גבוהה יותר)

לשלוט בסדר

סוגריים משנים את הסדר

רוצים ש Python-תחשב קודם את החיבור? **תעטפו אותו בסוגריים** — סוגריים תמיד ברמת העדיפות הכי גבוהה, בדיוק כמו במתמטיקה.

$(2 + 3) * 4$

20 — הסוגריים משנים את הסדר

$2 + 3 * 4$

14 — כפל קודם, כרגיל

דוגמה · סוגריים

אותם מספרים, סדר שונה

אותם שלושה מספרים, אותם אופרטורים — רק הסוגריים משתנים:

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

פלט:

```
14
20
```

תרגול סדר עדיפויות

נחשו לפני שמריצים

אל תריצו מיד — נמקו קודם על דף:

1. [בינוני] נחשו: מה תדפיס $2 * 3 - 10$? נמקו לפני שאתם בודקים.
2. [בינוני] הוסיפו סוגריים לביטוי $2 * 3 - 10$ כך שהתוצאה תהיה 24, ובדקו שזה עובד.
3. [בינוני] נחשו: מה התוצאה של $(4 + 6) / 2 - 1$?

מושג נוסף

הקשרים בין אופרטורים

מה קורה כשיש שני אופרטורים **באותה רמת עדיפות** זה לצד זה, כמו ב- $20 - 5 - 3$? כאן נכנס כלל **ההקשרים** (*associativity*) — הכיוון שבו Python קוראת "את הביטוי".

ימין לשמאל (רק **)

$2 ** 3 ** 2$ מחושב כמו $2 ** (3 ** 2)$ = 512

שמאל לימין (רוב האופרטורים)

$20 - 5 - 3$ מחושב כמו $(20 - 5) - 3 = 12$

דוגמה הקשרים

שני כיוונים שונים

אותה מבנה ביטוי — כיוון קריאה שונה, כי ל $**$ יש הקשר הפוך:

```
print(20 - 5 - 3)      # שמאל לימין
print(2 ** 3 ** 2)     # ימין לשמאל
```

פלט:

12
512

לא רק מספרים קבועים

ביטויים עם משתנים

ביטוי לא חייב להיות מורכב רק ממספרים קבועים — אפשר לשלב בו **משתנים** בדיוק כמו ערכים וגילים, ואפילו לשמור את התוצאה במשתנה חדש.

טיפ. Python קודם מחשבת את הביטוי בצד ימין של ה-`=`, ורק אז שומרת את התוצאה במשתנה בצד שמאל.

דוגמה

חישוב סכום קנייה

מחיר ליחידה, כמות — והתוצאה נשמרת במשתנה חדש:

```
price = 25
qty = 3
total = price * qty

print(total)
```

פלט:

75

תרגול חלק א'

משימות — רמה קלה

שילוב של אופרטורים ומשתנים, בלי סיבוכים.

1. [קל] צרו משתנים `price = 12.5` ו-`quantity = 4`, חשבו את סך ההכנסה (`price * quantity`) והדפיסו אותה.

2. [קל] בהינתן שלושה ציונים `g1`, `g2`, `g3` — חשבו את הממוצע שלהם בעזרת `(g1 + g2 + g3) / 3` והדפיסו.

3. [קל] צרו משתנים `width` ו-`height`, חשבו את שטח המלבן (`width * height`) והדפיסו.

מלכודת נפוצה

מה קורה בחילוק באפס?

מה קורה אם מנסים לחלק במספר אפס? בדיוק כמו במתמטיקה — זו פעולה לא חוקית, ו Python-עוצרת ומתריעה:

```
print(5 / 0)
```

```
ZeroDivisionError: division by zero
```

שימו לב. הכלל תקף גם ל-// וגם ל-%. כשנלמד תנאים ביחידה 6, נראה איך בודקים מראש שהמכנה לא אפס.

מלכודת נפוצה נוספת

/ מול // — לא לבלבל

שני האופרטורים נראים דומים, אבל מחזירים תוצאה שונה לגמרי:

```
7 // 2
```

3 — חילוק שלם, מחזיר int

```
7 / 2
```

3.5 — חילוק רגיל, מחזיר float

שימו לב. השתמשו ב-// רק כשאתם ממש רוצים תוצאה שלמה (למשל לחלק אנשים לקבוצות) — אחרת / הרגיל הוא מה שרוצים.

האזהרה החשובה מכולן

סדר לא נכון לא מתריע

שגיאת תחביר עוצרת את התוכנית ומתריעה. אבל **סדר פעולות שגוי הוא לא שגיאה בכלל** — התוכנית פשוט מחשבת את הדבר הלא נכון, בלי להתריע על כלום.

שימו לב. זו הסיבה שחשוב לבדוק ביטויים מורכבים בעזרת סוגריים ברורים — גם אם טכנית לא חובה, זה מונע טעויות שקטות שקשה למצוא אחר כך.

דוגמה מסכמת

עגלת קניות — הכל ביחד

מחיר, כמות, והנחה קבועה — כל מה שלמדנו היום בתוכנית קטנה אחת:

```
price = 49.9
qty = 3
discount = 10

total = price * qty - discount

print("מחיר סופי:", total)
```

פלט:

מחיר סופי: 139.69999999999998

תרגול חלק ב'

משימות —רמה בינונית

כאן נדרש שילוב של כמה אופרטורים באותו ביטוי.

1. [בינוני] בהינתן $n = 29$, בדקו האם הוא זוגי או אי-זוגי בעזרת $n \% 2$, והדפיסו את התוצאה עם תווית מסבירה.
2. [בינוני] צרו משתנה `seconds = 500` וחשבו כמה דקות שלמות וכמה שניות נשארות (בעזרת `//` ו-`%`), הדפיסו את שניהם.
3. [בינוני] נחשו לפני שמריצים: `x = 2 + 3 * 2 - 1` — מה יודפס ב-`print(x)`? נמקו לפי סדר העדיפויות.
4. [בינוני] חשבו את השטח וההיקף של מלבן שרוחבו 6 וגובהו `(width + height) * (width * height)` 9 (2, הדפיסו את שניהם עם תוויות).

תרגול חלק ג'

משימות —רמה מאתגרת

תרגילים שדורשים חשיבה על סדר העדיפויות וההקשרים יחד.

1. [מאתגר] טבלת מעקב (trace table) בהינתן `x = 2 + 3 * 4 - 1`, בלי להריץ — כתבו על דף שלב אחר שלב לפי סדר העדיפויות מה ערכו הסופי של `x`. אחר כך הריצו ובדקו את עצמכם.
2. [מאתגר] בנו "מחשבון טיפ למסעדה" (`bill`) סכום החשבון, `tip_percent` (0.15 ומכ `float` כ-פיט זוחא), וחשבו את הטיפ ואת הסכום הכולל לתשלום — הדפיסו את שניהם עם תוויות.
3. [מאתגר] איתור באג: מטרת הקוד לחשב ממוצע של שלושה ציונים, אבל התוצאה יוצאת שגויה כי חסרים סוגריים: `avg = 80 + 90 + 70 / 3` מצאו את הבעיה, תקנו אותה, והסבירו למה הקוד המקורי נתן תוצאה שגויה.

בדיקה עצמית

שאלות חזרה מהירות

בלי לגלול אחורה — כמה מהן אתם זוכרים?

1. [קל] מה ההבדל בין / ל-//?
2. [קל] מה מחזיר 2 % 7, ולמה?
3. [בינוני] מה סדר העדיפויות בין *, **, + ו-?
4. [בינוני] מה ההבדל בין הקשר "שמאל לימין" ל"ימין לשמאל", "ואיזה אופרטור שלמדנו היום הוא היוצא מן הכלל?

לפני שסוגרים

חמישה דברים לזכור מהיום

העלאה בחזקה — לא ^

// %
חילוק שלם, ושארית החילוק

+ - * /
ארבעת האופרטורים הבסיסיים — /
תמיד מחזיר float

שמאל לימין
ההקשר של רוב האופרטורים — חוץ
מ ** - שהוא ימין לשמאל

()
סוגריים תמיד מחושבים ראשונים,
וקובעים את הסדר

עכשיו התוכניות שלכם מחשבות בשבילכם

בשיעור הבא: עיצוב משתנים וכתובת קוד נקי — כללי שמות רשמיים והרגלים שהופכים קוד לקריא.